Day 11 Evidence Workbooklet

For Loops, Range Traces, Counters, and Accumulators

Engineering practice to repeated cases to loop variable to count and total to evidence note

Student copy. Guided workbooklet, not the mastery quiz. Print format: duplex, left-stapled packet.

Name		**Section**		**Date**	
Score	_____/49		Mastery check	secure / developing / needs support	

Concrete engineering situation

The sensor-kit checkout desk now has a batch of repeated checkout records. Week 2 could trace one kit at a time. Day 11 introduces a for loop so the same evidence move can repeat across a known number of kits.

Engineering task	Trace a known-size batch of sensor-kit records and summarize how many records need support.
Computational move	Use a loop variable to repeat the same action a fixed number of times, while updating a counter or total each pass.
Python focus	for, range, loop variable, iteration count, counter update, accumulator update, and final summary print.
Evidence to keep	range values, loop variable value, state before update, state after update, final count, final total, and off-by-one check.

Day 11 working rules

1. A for loop repeats once for each value produced by range.
2. range(4) produces 0, 1, 2, 3.
3. A counter usually changes by 1.
4. An accumulator usually adds a measured value.
5. Trace state after every iteration before trusting the final result.

Point map Manage your evidence blueprint

Points	Section	Evidence focus
1	Read a fixed-size loop before tracing it	recognize, predict
2	Trace the counter state after each pass	trace, state
3	Use range boundaries without off-by-one errors	boundary, explain
4	Separate counters from accumulators	accumulate, explain
5	Complete a small for-loop summary	implement, test
6	Transfer and support route	transfer, reflect

1 Read a fixed-size loop before tracing it

recognize predict

Start with a loop whose number of passes is known. The loop variable is evidence, not decoration.

```
support_count = 0

for kit_number in range(4):
    support_count = support_count + 1

print(support_count)
```

Pts.	Prompt	My evidence / response
1	What values does range(4) produce?	
1	How many times does the loop body run?	
1	What is the starting value of support_count?	
1	What final value is printed?	

2 Trace the counter state after each pass

trace state

Complete a state trace. Do not jump from the first line to the final answer.

```
support_count = 0
for kit_number in range(4):
    support_count = support_count + 1
```

Pts.	Prompt	My evidence / response
2	Pass for kit_number 0: value before update and after update.	
2	Pass for kit_number 1: value before update and after update.	
2	Pass for kit_number 2: value before update and after update.	
2	Pass for kit_number 3: value before update and after update.	
2	One-sentence final-count evidence note.	

Support route

A secure loop trace names the loop variable and the changing state variable for each pass.

3 Use range boundaries without off-by-one errors

boundary explain

Loops make boundary errors easy. Read the start, stop, and count before writing conclusions.

```
for day_index in range(1, 6):
    print("check day", day_index)
```

Pts.	Prompt	My evidence / response
2	What is the first value printed for day_index?	
2	What is the last value printed for day_index?	
2	How many lines are printed?	
2	Why does this loop not print day_index 6?	
2	Write a range call that would print 0, 1, 2, 3, 4.	

Common mistake to avoid

The stop value in range(start, stop) is not included.

4 Separate counters from accumulators

accumulate explain

A counter answers how many. An accumulator answers how much total. This page separates the two.

```
ready_count = 0
total_charge = 0

for charge_percent in [82, 77, 91]:
    total_charge = total_charge + charge_percent
    if charge_percent >= 80:
        ready_count = ready_count + 1

print(ready_count, total_charge)
```

Pts.	Prompt	My evidence / response
2	Which variable is the counter? What question does it answer?	
2	Which variable is the accumulator? What question does it answer?	
2	Trace ready_count after 82, 77, and 91.	
2	Trace total_charge after 82, 77, and 91.	
2	Write the exact final printed values.	

5 Complete a small for-loop summary

implement test

Complete the loop so it counts how many kit records are below the charge threshold.

```
low_charge_count = 0

for charge_percent in [76, 80, 95, 72]:
    if _______________________________:
        low_charge_count = low_charge_count + 1

print(low_charge_count)
```

Pts.	Prompt	My evidence / response
3	Fill in the condition.	
2	What final count should print?	
2	Which two charge values made the count change?	
2	What boundary test protects the exact value 80?	

6 Transfer and support route

transfer reflect

Close Day 11 by explaining why loops are useful only when the repeated evidence is still visible.

Pts.	Prompt	My evidence / response
4	Write a short note explaining how a for loop changes one-case checkout into repeated-case checkout.	
2	Circle your support route: range values / loop count / counter update / accumulator update / boundary case. Name one next action.	

Support route

Day 12 keeps the repeated-case idea but changes the stopping rule. Instead of a known number of passes, a while loop continues until a condition changes.